Exception Handling

1. Pendahuluan
2. Class-class Exception dan Class-class Error
3. Mengantisipasi Exception
4. Mekanisme Mengantisipasi Exception
5. Menampilkan Pesan Exception
6. Contoh Aplikasi Exception

Pendahuluan

- Error (kesalahan) dalam pemrograman dibagi dalam tiga katagori yaitu ***syntax error*** (terjadi saat kompilasi), ***run time error*** (saat program dieksekusi), dan ***logic error*** (ketika output belum sesuai dengan yang diharapkan).
- Exception digunakan sebagai sarana untuk ***melaporkan*** jenis *run time error* dan ***mengendalikan***nya agar *run time error* tersebut tidak mengakibatkan eksekusi dihentikan (statement setelahnya tetap dieksekusi).
- Dalam java, *exception* merupakan objek dari *subclass Exception* dari *super class Throwable*. *Class Throwable* ini terdapat dalam *package java.lang.object*.

class Error dan class Exception

- Kejadian yang terdeteksi sebagai objek dari *class Error* dan turunannya tergolong fatal, sehingga sistem tidak dapat dimanipulasi sehingga terkendalikan. Contoh class standard jenis Error misalnya *LinkageError*, *VirtualMachineError*, dan *AWTError*.
- Kejadian yang terdeteksi sebagai objek dari *class standard* jenis *Exception* dan turunannya masih dapat diantisipasi dengan menyisipkan statement tambahan untuk mendeteksi penyebab dan jenisnya.
- Kelompok objek dalam *class RuntimeException* diperiksa oleh interpreter apakah akan ditangani atau dilempar, namun ada pula exception yang tidak diperiksa oleh interpreter.
- Programmer dibolehkan membuat sendiri *class non standard* dengan cara ***extends*** dari class standard *Exception*.

Checked Exception

No	Exception	Deskripsi
1	ClassNotFoundException	Class tidak ditemukan
2	CloneNotSupportedException	Melakukan clone objek yang tidak mengimplementasikan interface Cloneable
3	IllegalAccessException	Akses ke suatu class ditolak
4	InstantiationException	Menciptakan objek dari abstract class atau dari interface
5	InterruptedException	Thread telah diinterupsi oleh thread lain
6	NoSuchFieldException	Field yang diminta tidak ada
7	NoSuchMethodException	Method() yg diminta tidak ada

Uncheked Exception

NO	Exception	Deskripsi
1	ArithmaticException	Kesalahan proses arithmatic misalnya pembagian dengan bilangan nol
2	ArrayIndexOutBoundsException	Index arrray di luar batas
3	ArrayStoreException	Pemberian nilai ke elemen array tidak sesuai dengan tipenya
4	ClassCastException	Cast yang tidak sah
5	IllegalArgumentException	Argument Illegal
6	IllegalMonitorStateException	Operasi monitor illegal seperti menunggu di thread yang tidak terkunci
7	IllegalStateException	Lingkungan aplikasi state yang tidak benar
8	IllegalThreadStateException	Operasi yg diminta tidak kompatibel dengan state thread saat itu

Uncheked Exception

NO	Exception	Deskripsi
9	IndexOutOfBoundsException	Index di luar batas
10	NegativeArraySizeException	Array diciptakan dengan ukuran negatif
11	NullPointerException	Penggunaan null yang tidak sah
12	NumberFormatException	Konversi yg tidak sah dari string ke format numerik
13	SecurityException	Melanggar aturan security
14	StringIndexOutOfBounds	Index di luar batas string
15	UnsupportedOperationException	Ditemukan operasi yang tidak didukung

Mangantisipasi Exception

- Diperlukan dua langkah untuk mengantisipasi exception:
- 1. Mendeklarasikan Exception
- Melempar Exception
- 2. Menangkap Exception

Dilakukan di method() di Class-class

Mendeklarasikan Exception (di Method())

- Bentuk umum:
- [modifier] returnType namaMethod() throws ExceptionClass { //deklarasi
-
- //ada statement melempar exception
- ExceptionClass ec = new ExceptionClass();
- throw ec;
- }

- Contoh1:
- public abstract beriPinjaman(int vduit) throws TolakException; //deklarasi
- **ClassNotFoundException** adalah class standard, sedangkan **TolakException** adalah class non standard yang diturunkan dari class standard.

Contoh2

- public string operasiMatematika() throws ClassNotFoundException { // deklarasi{
- //melempar
- ClassNotFoundExceptioin cnfe = new ClassNotFoundException();
- **throw cnfe;**
- }

Contoh3

- public void beriPinjaman() throws TolakException { //deklarasi
-
- //ada statement melempar exception
- TolakException tolak = new TolakException();
- throw tolak;
- }

Melempar Exception

- Di dalam method() yang telah dideklarasikan, tepatnya di bawah statemtent yang akan dikendalikan.
- Bentuk umum:
- **ExceptionClass namaObjek = new ExceptionConstructor();**
- **throw namaObjek;**

- Diringkas menjadi:
- **throw namaObjek ExceptionConstructor();**

- Atau menjadi:
- **throw new ExceptionConstructor();**

- Contoh:
- **TolakException t = new TolakException("lagi pelit!");**
- **throw t;**
- Diringkas menjadi:
- **throw new TolakException("lagi pelit!");**

Menangkap Exception

- Penangkapan runtime error dengan cara membuat beberapa blok di dalam method tersendiri (bukan method yg mendeklarasikan jenis exception) di method yg memanggil misal method main(), di Main Class, yg bentuk umumnya sebagai-berikut:
- **try {**
- **//pemanggilan method telah dideklarasikan melempar exception**
- **//yg mungkin terjadi exception**
- **//statement yg mungkin terjadi exception**
- **}**
- **catch (JenisException1 namaObjek) { //penangkapan objek exception**
- **//penanganan bila terjadi JenisException1**
- **}**
- **catch (JenisException2 namaObjek) { //penangkapan objek exception**
- **//penanganan bila terjadi JenisException2**
- **}**
- **finally {**
- **//statements yg wajib dieksekusi**
- **}**
- Blok di atas menangkap objek dari salah-satu class Exception yg dilempar dari throw/throws dan/atau blok try.

Mekanisme Mengantisipasi Exception

Urutan jenis Exception mana yg wajib didahulukan dalam blok catch yaitu mulai dari Exception yang **paling khusus** diletakkan pada blok catch **paling atas** dan seterusnya Exception yang **paling umum** diletakkan pada blok catch **paling bawah.**

Ada tiga skenario exception:

1. Jika **tidak terjadi** exception (tidak ada kode dalam blok catch yang dieksekusi).
2. Jika exception **terjadi** pada blok method() tunggal (salah-satu blok catch dieksekusi).
3. Jika **terjadi** exception pada blok bersarang.

Pada semua skenario, blok finally tetap wajib dieksekusi.

Exception pada blok bersarang

```
pertama() {
    ...
    try {
        ...
        //memanggil method kedua()
    }
    catch(Exception1 obj1) {
        //proses thd objek1;
    }
    ...
}
```

```
kedua() {
    ...
    try {
        ...
    }
    catch(Exception2 obj2) {
        //proses thd objek2;
    }
    ...
}
```

Menampilkan Pesan Exception

- Beberapa method() standard dapat digunakan untuk menampilkan pesan exception, yang merupakan turunan dari class standard java.lang.Throwable.

Method untuk menampilkan pesan Exception

No	Method() pesan exception	Deskripsi
1	getMessage()	**Mengembalikan nilai string** yang berisi pesan tentang objek kejadian exception dari class standard Throwable.
2	toString()	**Mengembalikan nilai string** yang berisi pesan singkat tentang objek dari suatu kejadian exception.
3	getLocalizedMessage()	**Menampilkan** pesan exception lokal yang terjadi di dalam subclass saja.
4	printStackTrace()	Method() ini bersifat void, dan hanya **mencetak** informasi tentang objek dari class standard Throwable.

```
public class BagiNol {
    public static void main(String[] args) {
        System.out.println("Sebelum pembagian");
        System.out.println(5/0);
        System.out.println("Sesudah pembagian");
    }
}
```

Sebelum pembagian

Exception in thread “main” java.lang.ArithmeticException: / by zero at BagiNo1.main<BagiNo1.java:6>

```
public class BagiNol {
 public static void main(String[] args) {
   System.out.println("Sebelum pembagian");
   try {
     System.out.println(5/0);
   }
   catch (Exception a) {
     System.err.println("Terjadi pembagian dengan nol");
      System.err.println("Pesan dari Sun: " +a.getMessage());
   }
   System.out.println("Sesudah pembagian");
  }
}
```

```
Sebelum pembagian

Terjadi pembagian dengan nol

Pesan dari Sun: / by zero
```

```
public class TolakException extends Exception {
  public TolakException(String pesan) {
      super(pesan);
  }
}
public interface Kreditor {
  public void beriPinjaman(int vduit) throws TolakException;
}
public class Orang {
  protected String nama;
  public Orang(String vnama) {
      nama = vnama;
  }
  public String getNama() {
      return nama;
  }
  public void ngomong(String message) {
      System.out.println(nama + " ngomong: " + message);
  }
}
```

```
public class Bank implements Kreditor {
   private Orang manager;
   private int asset;
   public Bank(String vmanager, int vasset) {
     manager = new Orang(vmanager);
     asset = vasset;
   }
   public void beriPinjaman(int vduit) throws TolakException {
     if (vduit < 1000000) {
        asset = asset - vduit;
        manager.ngomong("Bank memberikan pinjaman Rp " +vduit);
        manager.ngomong("Asset bank sekarang Rp " + asset);
     }
     else {
        manager.ngomong("Bank tidak akan memberikan pinjaman!");
        throw new TolakException("Bank lagi pelit!");
     }
   }
}
```

```
public class OrangKaya extends Orang implements Kreditor {
   private int duit;
   public OrangKaya(String vnama, int vduit) {
       super(vnama);
       duit = vduit;
   }
   public void beriPinjaman(int vduit) throws TolakException{
     if (vduit < 500000) {
         duit = duit - vduit;
         super.ngomong("Saya memberikan pinjaman Rp " + vduit);
         super.ngomong("Duit saya sekarang Rp " + duit);
     }
     else {
         super.ngomong("Saya tidak akan memberikan pinjaman!");
         TolakException t = new TolakException("Orang Kaya lagi peli
         throw t;
     }
   }
}
```

```
public class OrangMiskin extends Orang {
   private int utang;
   public OrangMiskin(String vnama, int vutang) {
     super(vnama);
     utang = vutang;
   }
   public void pinjamDuit(Kreditor vkreditor, int vduit) {
     try {
         vkreditor.beriPinjaman(vduit);
         utang = utang + vduit;
         this.ngomong("Saya mendapatkan pinjaman Rp " + vduit);
         this.ngomong("Utang saya sekarang Rp " + utang);
     }
     catch (TolakException e) {
         System.err.println(e.getMessage());
         this.ngomong("Malangnya nasibku, saya telah ditolak!");
       }
   }
}
```

```
public class PinjamApp {
  public static void main(String args[]) {
      Bank bankRut = new Bank("Cesar", 5000000);
      OrangKaya amir = new OrangKaya("Amir", 2000000);
      OrangMiskin badu = new OrangMiskin("Badu", 1500000);
      try {
        badu.pinjamDuit(amir, 600000);
        badu.pinjamDuit(bankRut, 800000);
      }
      catch (Exception e) {
        System.out.println("Nggak pernah jalan!");
      }
  }
}
```